



SOICT

BIG DATA STORAGE AND PROCESSING

COURSE: IT4043E

NYC Taxi Data Analytics Platform

Author

DAO VU TIEN DAT
TRAN HUU DAO
NGO QUANG DUC
VU NHU DUC
TA TUAN HAI

Mentor

PHD. TRAN VIET TRUNG

December 2025

Contents

1. Introduction	3
1.1 Problem	3
1.2 Motivation	3
2. Data Model	3
3. Technologies Used and System Architecture	4
3.1 System Architecture	4
3.2 Key Technologies	5
4. System Component Details	6
4.1 Postgresql	6
4.1.1 PostgreSQL Configuration	6
4.1.2 Database Schema	7
4.1.3 Data Ingestion Simulator	8
4.2 Debezium (Change Data Capture)	9
4.2.1 The Need for Change Data Capture	9
4.2.2 Why Debezium	10
4.2.3 Debezium Connect Configuration	11
4.2.4 PostgreSQL Source Connector	11
4.2.5 CDC Data Flow	12
4.3 Apache Kafka	13
4.3.1 Kafka Cluster Configuration	13
4.3.2 Debezium CDC Integration	13
4.4 Minio	13
4.4.1 MinIO Cluster Configuration	14
4.4.2 Spark Integration	14
4.5 Spark	15
4.5.1 Spark Thrift Server (JDBC/ODBC Gateway)	15
4.5.2 Spark Thrift Server Configuration	15
4.5.3 Entrypoint and Dependency Management	16
4.5.4 Spark Storage Configuration	16
4.5.5 Spark Medallion Processing	17
4.6 Flink	19
4.6.1 Flink Cluster Configuration	19
4.6.2 Streaming Job: Kafka (CDC) → Flink (KPIs) → Cassandra	19
4.7 Cassandra	20
4.7.1 Schema and Tables	21
4.8 Airflow	22
4.8.1 Airflow Deployment Architecture	22
4.8.2 Spark Connection Configuration	23

4.8.3	Lakehouse Pipeline DAG	24
4.8.4	Monitoring and Observability	25
4.9	Streamlit	26
4.10	Data Visualization with Streamlit and PyHive	26
4.10.1	Dashboard Containerization	26
4.10.2	PyHive Connection to Spark Thrift Server	27
4.10.3	Python Dependencies	27
4.10.4	Business Intelligence Dashboard	28
4.10.5	Chart Visualizations with Plotly	31
4.11	Prometheus + Grafana	32
4.11.1	Prometheus Configuration	32
4.11.2	Spark Metrics Exporter	33
4.11.3	Grafana Visualization	33
4.11.4	Spark Cluster Dashboard	34
4.11.5	Monitoring Use Cases	36
5.	Conclusion	36
6.	Acknowledgments	37
6.1	Lessons Learned	37
6.1.1	Lesson 1: Container Networking & Listener Configuration	37
6.1.2	Lesson 2: Checkpointing, Watermarks, and Exactly-once Safety	37
6.1.3	Lesson 3: Schema Evolution with Iceberg	37
7.	Member Contribution	38

1. Introduction

1.1 Problem

This project focuses on designing and implementing a robust Kappa-style big data pipeline capable of supporting accurate weather forecasting and near real-time monitoring. While modern IoT weather stations generate vast amounts of environmental data, this raw data is often incomplete, noisy, and unstructured, making it unsuitable for direct predictive modeling.

The core problem is to bridge the gap between raw sensor ingestion and reliable machine learning predictions. This requires a system that not only stores data but actively repairs it – handling missing timestamps, smoothing outliers, and engineering "lag features" to create a high-quality dataset. The system aims to automate the end-to-end flow from data collection to the generation of training datasets required for forecasting models.

1.2 Motivation

Urban transportation systems generate massive volumes of heterogeneous data that reflect complex interactions between human mobility, economic activity, and infrastructure constraints. Among publicly available transportation datasets, the New York City taxi trip records released by the **contentReference[oaicite:0]index=0** provide a particularly rich and realistic setting for large-scale data analytics. The dataset contains detailed trip-level information, including temporal attributes, spatial locations, trip distances, durations, and fare components, enabling fine-grained analysis of mobility and pricing behavior in a dense metropolitan environment.

From a big data perspective, NYC taxi data naturally embodies the defining characteristics of modern data systems. The scale of the data introduces challenges related to distributed storage, efficient querying, and scalable computation, while the temporal and spatial diversity of trips requires careful feature engineering and aggregation strategies. As a result, this dataset serves as an effective benchmark for designing and evaluating end-to-end data pipelines that integrate ingestion, transformation, analytics, and visualization within a unified platform.

Beyond technical considerations, taxi trip data provides meaningful analytical value for understanding demand patterns, revenue drivers, and spatio-temporal dynamics in urban transportation. Analyzing how trip volume, distance, duration, and fares vary across locations and time periods allows stakeholders to reason about peak demand, pricing variability, and operational efficiency. These insights are directly relevant to real-world decision-making contexts such as transportation planning, dynamic pricing analysis, and policy evaluation.

Motivated by these factors, the NYC Taxi Data Analytics Platform is designed to bridge scalable data engineering with applied analytics. By transforming raw trip records into structured, queryable features and interactive analytical outputs, the project demonstrates how large-scale urban mobility data can be systematically leveraged to support exploratory analysis, modeling, and data-driven insights in a realistic big data setting.

2. Data Model

We will use the data schema following New York City Taxi and Limousine Commission (NYC TLC) Trip Record Data.

Table 1 summarizes the key variables obtained from the NYC TLC Trip Record Data, providing detailed information on trip characteristics, pricing components and spatial identifiers.

Table 1: NYC TLC Trip Record Columns

Column	Description
id	Trip ID (Primary Key)
tpep_pickup_datetime	Date and time when the taximeter was engaged (trip start).
tpep_dropoff_datetime	Date and time when the taximeter was disengaged (trip end).
passenger_count	Number of passengers reported for the trip.
trip_distance	Elapsed trip distance in miles as reported by the taximeter.
payment_type	Method of payment. (0 = Flex Fare; 1 = Credit card; 2 = Cash; 3 = No charge; 4 = Dispute; 5 = Unknown; 6 = Voided trip).
fare_amount	Time-and-distance fare calculated by the taximeter, excluding surcharges.
tip_amount	Tip amount recorded for credit card payments (cash tips not included).
total_amount	Total amount charged to the passenger, excluding cash tips.
pickup_longitude	Longitude of pickup location
pickup_latitude	Latitude of pickup location
dropoff_longitude	Longitude of dropoff location
dropoff_latitude	Latitude of dropoff location

3. Technologies Used and System Architecture

3.1 System Architecture

The NYC Taxi Data Analytics Platform is designed following a **modern event-driven data architecture**, enabling scalable ingestion, reliable storage, and flexible analytical processing across both real-time and historical data. The architecture separates transactional workloads from analytical workloads while maintaining strong data consistency through change data capture (CDC) and streaming pipelines.

As illustrated in Figure 2, the system is organized into four logical layers—OLTP, Data Ingestion, Data Storage, and Analytics—supported by a unified compute engine.

- **OLTP Layer (Source Systems):** Operational taxi trip data is stored in **PostgreSQL**, which serves as the transactional database for high-frequency inserts and updates. This layer represents the authoritative source of raw trip records.
- **Data Ingestion Layer (CDC & Streaming):** Changes from PostgreSQL are captured using **Debezium**, enabling log-based change data capture without impacting transactional performance. These change events are published to **Apache Kafka**, which functions as a durable, ordered event log and decouples data producers from downstream consumers.
- **Data Storage Layer (Lakehouse & Serving Stores):** Streaming data from Kafka is consumed by **Apache Spark**, which performs both real-time and batch transformations. Processed data is written to:
 1. A **Lakehouse** built on **HDFS** using open table formats (e.g., Apache Iceberg), supporting the Medallion Architecture (Bronze, Silver, Gold) for progressive data refinement.
 2. **Apache Cassandra**, which serves as a low-latency NoSQL store optimized for real-time and

near real-time query access.

Workflow scheduling and dependency management for batch transformations are orchestrated using **Apache Airflow**, ensuring reproducible and reliable pipeline execution.

- **Analytical Layer:** Analytical queries and aggregations are executed using **Apache Spark** on curated Gold-layer datasets. Business intelligence and exploratory analysis are delivered through **Apache Streamlit**, providing interactive dashboards and visual insights for stakeholders.

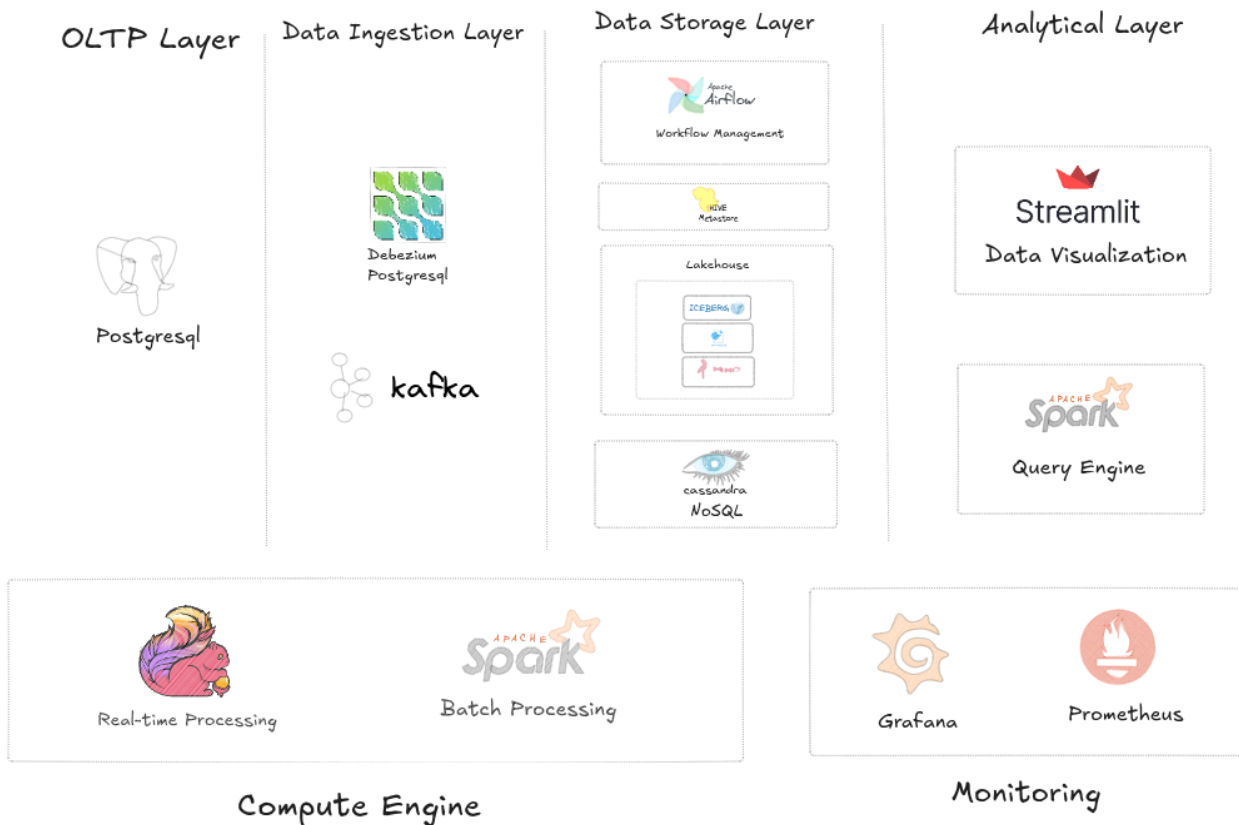


Figure 1: NYC Taxi Data Analytics Platform System Architecture

3.2 Key Technologies

The platform is fully containerized to ensure portability, scalability, and environment consistency. The primary technologies are summarized below.

- **Docker**
 - **Function:** Containerizes all system components, including databases, streaming services, and analytics tools.
 - **Benefits:** Simplifies deployment, improves reproducibility, and minimizes environment-related inconsistencies.
- **PostgreSQL**
 - **Function:** Acts as the OLTP database for taxi trip records.
 - **Benefits:** Provides transactional guarantees and efficient handling of structured relational data.
- **Debezium**
 - **Function:** Captures database changes from PostgreSQL using CDC.

- **Benefits:** Enables real-time data propagation with minimal impact on source systems.
- **Apache Kafka**
 - **Function:** Serves as the central streaming backbone for all change events.
 - **Benefits:** Ensures fault tolerance, message durability, and scalable data distribution.
- **Apache Flink**
 - **Function:** Performs stateful, low-latency stream processing on events from Kafka.
 - **Benefits:** Provides exactly-once processing guarantees, event-time semantics, and scalable real-time analytics.
- **Apache Spark**
 - **Function:** Processes streaming and batch data for transformation, aggregation, and analytics.
 - **Benefits:** Supports scalable distributed computation and unified processing across workloads.
- **Apache Iceberg**
 - **Function:** Stores historical and curated datasets in columnar formats.
 - **Benefits:** Enables efficient analytical queries, schema evolution, and large-scale batch processing.
- **Apache Cassandra**
 - **Function:** Stores real-time aggregates and metrics for fast retrieval.
 - **Benefits:** Provides high availability and low-latency reads at scale.
- **Apache Airflow**
 - **Function:** Orchestrates batch workflows and Spark job scheduling.
 - **Benefits:** Improves pipeline reliability and simplifies dependency management.
- **Streamlit**
 - **Function:** Provides data visualization and dashboarding capabilities.
 - **Benefits:** Enables interactive exploration and communication of analytical results.

4. System Component Details

4.1 Postgresql

PostgreSQL serves as the operational database in our Taxi Big Data Platform, acting as the primary transactional system for NYC taxi trip data. It stores real-time trip records and serves as the source of truth for all downstream processing and analytics.

4.1.1 PostgreSQL Configuration

Our PostgreSQL instance is deployed using Docker containers with version 16-alpine for a lightweight footprint. The database is configured with the following parameters:

```

1 db:
2   image: postgres:16-alpine
3   container_name: postgres
4   restart: always
5   environment:
6     POSTGRES_USER: admin
7     POSTGRES_PASSWORD: 123
8     POSTGRES_DB: nyc_taxi_db

```

```

9  ports:
10   - "5432:5432"
11  volumes:
12   - db_data:/var/lib/postgresql/data
13   - ./configs/postgresql.conf:/etc/postgresql/postgresql.conf
14  command:
15   - "postgres"
16   - "-c"
17   - "wal_level=logical"
18   - "-c"
19   - "max_wal_senders=1"
20   - "-c"
21   - "max_replication_slots=1"
22  healthcheck:
23   test: ["CMD-SHELL", "pg_isready -U admin -d nyc_taxi_db"]
24   interval: 10s
25   timeout: 5s
26   retries: 5

```

The key configuration parameters are specified via command-line flags and also in the mounted `postgresql.conf` file:

```

1  # Write Ahead Log (WAL) settings
2  wal_level = logical
3  max_wal_senders = 1
4  max_replication_slots = 1
5
6  # Connection and performance settings
7  max_connections = 100
8  shared_buffers = 128MB
9
10 # Logging for debugging and monitoring
11 log_statement = 'mod'
12 log_line_prefix = '%t [%p]: [%l-1] user=%u,db=%d,app=%a,client=%h '

```

The `wal_level=logical` setting enables logical replication capabilities, allowing the database to support downstream Change Data Capture (CDC) mechanisms. The healthcheck ensures the database is ready before dependent services start, preventing connection failures during system initialization.

4.1.2 Database Schema

The core operational table is `taxi_trips`, which stores NYC taxi trip records. The schema captures temporal, geospatial, and transactional dimensions of each trip:

```

1 CREATE TABLE IF NOT EXISTS taxi_trips (
2   id SERIAL PRIMARY KEY,
3   tpep_pickup_datetime TIMESTAMP,
4   tpep_dropoff_datetime TIMESTAMP,
5   passenger_count INTEGER,
6   trip_distance FLOAT,
7   pickup_longitude FLOAT,
8   pickup_latitude FLOAT,
9   dropoff_longitude FLOAT,
10  dropoff_latitude FLOAT,
11  fare_amount FLOAT,
12  tip_amount FLOAT,
13  total_amount FLOAT,
14  payment_type INTEGER
15 );

```

The schema includes:

- **Temporal Fields:** `tpep_pickup_datetime` and `tpep_dropoff_datetime` capture the start and end timestamps of each trip, enabling time-series analysis and trip duration calculations.
- **Geospatial Fields:** Longitude and latitude coordinates for both pickup and dropoff locations enable spatial analytics, heat mapping, and route analysis.
- **Transactional Fields:** `fare_amount`, `tip_amount`, and `total_amount` support revenue analytics and pricing models.
- **Categorical Fields:** `passenger_count` and `payment_type` enable segmentation and behavioral analysis.

4.1.3 Data Ingestion Simulator

Since we do not have access to live NYC taxi trip feeds, we developed a Python-based data generator (`insert_taxi_data.py`) that simulates real-time taxi operations. This script continuously inserts synthetic trip records into PostgreSQL, replicating the behavior of a production OLTP system.

The simulator generates realistic NYC taxi trips with the following characteristics:

```

1 # Database connection parameters
2 DB_PARAMS = {
3     'dbname': 'nyc_taxi_db',
4     'user': 'admin',
5     'password': '123',
6     'host': 'localhost',
7     'port': '5432'
8 }
9
10 def generate_random_location():
11     """Generate random NYC-like coordinates"""
12     return (
13         random.uniform(-74.03, -73.75), # longitude
14         random.uniform(40.63, 40.85)    # latitude
15     )
16
17 def generate_trip_data(num_records=1000):
18     """Generate simulated taxi trip data"""
19     PAYMENT_TYPES = [0, 1, 2, 3, 4, 5, 6]
20     for _ in range(num_records):
21         pickup_time = datetime.now()
22         dropoff_time = pickup_time + timedelta(
23             minutes=random.randint(10, 120)
24         )
25
26         pickup_long, pickup_lat = generate_random_location()
27         dropoff_long, dropoff_lat = generate_random_location()
28
29         trip_distance = random.uniform(0.5, 20.0)
30         fare_amount = 2.50 + (trip_distance * 2.50)
31         tip_amount = fare_amount * random.uniform(0, 0.3)
32         total_amount = fare_amount + tip_amount
33
34         payment_type = random.choice(PAYMENT_TYPES)
35
36         yield (
37             pickup_time, dropoff_time,
38             random.randint(1, 6), # passenger_count
39             round(trip_distance, 2),
40             pickup_long, pickup_lat,
41             dropoff_long, dropoff_lat,

```

```

42         round(fare_amount, 2),
43         round(tip_amount, 2),
44         round(total_amount, 2),
45         payment_type
46     )

```

The generator runs in a continuous loop, inserting batches of records at configurable intervals (default: 10 records every 5 seconds). This controlled ingestion rate allows us to simulate realistic operational load and validate the performance of downstream components:

```

1 def generate_continuous_data(interval_seconds=5, batch_size=10):
2     """Continuously generate and insert data at intervals"""
3     conn = create_connection()
4     try:
5         create_table(conn)
6         # Insert historical data first (baseline)
7         insert_data_past(conn, num_records=1000)
8
9         # Continuous real-time insertion
10        while True:
11            insert_data(conn, batch_size)
12            time.sleep(interval_seconds)
13    except KeyboardInterrupt:
14        conn.close()
15        print("Data generation stopped by user.")

```

The simulator first seeds the database with 1,000 historical records (with timestamps spanning the past 30 days) to establish a baseline dataset. It then continuously inserts new "real-time" records with current timestamps, producing approximately 2 records per second. This two-phase approach enables testing of both batch snapshot processing and incremental stream processing patterns in downstream pipelines.

4.2 Debezium (Change Data Capture)

4.2.1 The Need for Change Data Capture

In modern data architectures, maintaining synchronization between operational databases and analytical systems presents a fundamental challenge. Traditional approaches to data integration include:

- **Batch ETL (Extract-Transform-Load):** Periodically querying the source database to extract all records or records modified since the last extraction. This approach suffers from high latency (typically hours), significant database load during extraction windows, and difficulty in capturing deleted records.
- **Polling-Based Replication:** Continuously querying the database for changes using timestamp columns or sequence numbers. While offering lower latency than batch ETL, this method still imposes query load on the operational database, may miss rapid updates between polling intervals, and cannot reliably capture DELETE operations without additional audit tables.
- **Application-Level Dual Writes:** Modifying application code to write to both the operational database and the messaging system simultaneously. This approach introduces tight coupling, risks data inconsistency if one write fails, and requires changes to all data-producing applications.

Change Data Capture (CDC) addresses these limitations by reading directly from the database's transaction log—the authoritative record of all committed changes. This log-based approach offers several critical advantages for our taxi analytics platform:

1. **Zero Impact on Source Database:** CDC reads from the transaction log asynchronously, imposing no additional query load on PostgreSQL. This is essential for our high-throughput taxi trip ingestion where the operational database must maintain low-latency INSERT performance.
2. **Complete Change Capture:** Every INSERT, UPDATE, and DELETE is captured, including intermediate states. For taxi analytics, this means we can track fare adjustments, trip cancellations, and payment corrections that would be invisible to polling-based approaches.
3. **Low Latency:** Changes are propagated to Kafka within milliseconds of being committed, enabling near real-time analytics dashboards and streaming aggregations.
4. **Exactly-Once Semantics:** By maintaining replication slots and offset tracking, CDC ensures no events are lost or duplicated, providing the data consistency guarantees required for accurate revenue reporting.
5. **Schema Evolution Support:** CDC captures schema changes alongside data changes, allowing the analytical layer to adapt to evolving data models without pipeline modifications.

4.2.2 Why Debezium

Debezium is an open-source distributed platform purpose-built for change data capture. Among available CDC solutions, Debezium stands out for several reasons:

- **Native Kafka Integration:** Built on Kafka Connect, Debezium leverages Kafka's fault tolerance, scalability, and exactly-once delivery guarantees. This seamless integration eliminates the need for custom glue code between the CDC engine and our streaming infrastructure.
- **PostgreSQL-Native Protocol:** Debezium uses PostgreSQL's built-in logical replication (pgoutput plugin), avoiding the need for external plugins or triggers that could impact database performance or require elevated privileges.
- **Snapshot and Streaming Modes:** Debezium can perform an initial consistent snapshot of existing data before switching to streaming mode, ensuring our analytical layer captures both historical taxi trips and new real-time events.
- **Rich Transformation Capabilities:** Single Message Transforms (SMTs) allow event restructuring, filtering, and enrichment within the connector, reducing downstream processing complexity.
- **Production-Grade Reliability:** Debezium maintains replication slots and offset tracking to guarantee no data loss across connector restarts, network partitions, or Kafka broker failures.

In our pipeline, Debezium monitors the PostgreSQL transaction log (WAL) and streams every INSERT, UPDATE, and DELETE operation on the `taxi_trips` table to Apache Kafka in real-time. This architecture ensures that our analytical layer—including the Bronze Iceberg tables and real-time Flink aggregations—remains synchronized with the operational database without impacting transactional performance.

4.2.3 Debezium Connect Configuration

We deploy Debezium using the official Kafka Connect image with the PostgreSQL connector pre-installed. The connector runs as a distributed service, storing its offsets and configuration in dedicated Kafka topics:

```
1 debezium:
2   image: debezium/connect:2.5
3   container_name: debezium
4   restart: always
5   ports:
6     - "8083:8083"
7   environment:
8     BOOTSTRAP_SERVERS: kafka:29092
9     GROUP_ID: debezium-group
10    CONFIG_STORAGE_TOPIC: debezium_configs
11    OFFSET_STORAGE_TOPIC: debezium_offsets
12    STATUS_STORAGE_TOPIC: debezium_statuses
13    CONFIG_STORAGE_REPLICATION_FACTOR: 1
14    OFFSET_STORAGE_REPLICATION_FACTOR: 1
15    STATUS_STORAGE_REPLICATION_FACTOR: 1
16    KEY_CONVERTER: org.apache.kafka.connect.json.JsonConverter
17    VALUE_CONVERTER: org.apache.kafka.connect.json.JsonConverter
18    CONNECT_KEY_CONVERTER_SCHEMAS_ENABLE: "false"
19    CONNECT_VALUE_CONVERTER_SCHEMAS_ENABLE: "false"
20   depends_on:
21     kafka:
22       condition: service_healthy
23     db:
24       condition: service_healthy
```

Key configuration highlights:

- **Bootstrap Servers:** Points to our Kafka broker for publishing change events.
- **Storage Topics:** Debezium uses three internal topics to persist connector configuration, offset tracking, and status information.
- **JSON Converters:** We disable schema embedding to produce lightweight JSON payloads that are easier to parse downstream.

4.2.4 PostgreSQL Source Connector

The PostgreSQL connector configuration defines how Debezium captures changes from the `taxi_trips` table. We deploy this configuration automatically using a one-shot init container:

```
1 debezium-init:
2   image: curlimages/curl:8.8.0
3   container_name: debezium-init
4   restart: "no"
5   depends_on:
6     debezium:
7       condition: service_healthy
8   volumes:
9     - ./configs:/configs:ro
10  command:
11    ["--silent", "--show-error", "--retry", "30", "--retry-delay", "2",
12     "-X", "POST", "-H", "Content-Type: application/json",
13     "--data", "@/configs/debezium-connector.json",
14     "http://debezium:8083/connectors"]
```

The connector configuration file (`debezium-connector.json`) specifies the source database connection and CDC behavior:

```
1 {
2   "name": "taxi-postgres-connector",
3   "config": {
4     "connector.class": "io.debezium.connector.postgresql.PostgresConnector",
5     "database.hostname": "db",
6     "database.port": "5432",
7     "database.user": "admin",
8     "database.password": "123",
9     "database.dbname": "nyc_taxi_db",
10    "database.server.name": "taxi_db_server",
11    "table.include.list": "public.taxi_trips",
12    "plugin.name": "pgoutput",
13    "publication.autocreate.mode": "all_tables",
14    "slot.name": "debezium_taxi_slot",
15    "topic.prefix": "taxi",
16    "key.converter": "org.apache.kafka.connect.json.JsonConverter",
17    "value.converter": "org.apache.kafka.connect.json.JsonConverter",
18    "key.converter.schemas.enable": "false",
19    "value.converter.schemas.enable": "false",
20    "transforms": "unwrap",
21    "transforms.unwrap.type": "io.debezium.transforms.ExtractNewRecordState",
22    "transforms.unwrap.drop.tombstones": "false",
23    "transforms.unwrap.delete.handling.mode": "rewrite",
24    "transforms.unwrap.add.fields": "op,source.ts_ms",
25    "snapshot.mode": "initial",
26    "time.precision.mode": "connect"
27  }
28 }
```

Critical configuration parameters:

- **plugin.name = pgoutput:** Uses PostgreSQL's native logical replication protocol, which is more efficient than the legacy `decoderbufs` plugin.
- **table.include.list:** Limits CDC to only the `taxi_trips` table, reducing unnecessary overhead.
- **slot.name:** Creates a dedicated replication slot to ensure no events are lost between connector restarts.
- **topic.prefix:** All change events are published to topics prefixed with `taxi`, resulting in topics like `taxi.public.taxi_trips`.
- **transforms.unwrap:** The `ExtractNewRecordState` transformation flattens Debezium's envelope format, extracting only the "after" state of each record for simpler downstream processing.
- **snapshot.mode = initial:** On first startup, Debezium performs a full snapshot of existing data before switching to streaming mode.

4.2.5 CDC Data Flow

The change data capture pipeline follows this flow:

1. **WAL Capture:** PostgreSQL writes all transactions to the Write-Ahead Log with `wal_level=logical`.
2. **Replication Slot:** Debezium reads from the dedicated replication slot, ensuring exactly-once delivery semantics.
3. **Event Transformation:** The `ExtractNewRecordState` transform unwraps the change event envelope.
4. **Kafka Publication:** Transformed events are published to the `taxi.public.taxi_trips` topic.

5. **Downstream Processing:** Spark Structured Streaming or Flink consumes events for real-time analytics.

4.3 Apache Kafka

The Kafka cluster is deployed inside Docker and is configured as a single KRaft broker. Debezium Connect captures changes from PostgreSQL and publishes them to Kafka topics; a Spark streaming job consumes the CDC topics and writes into Iceberg tables.

4.3.1 Kafka Cluster Configuration

For local, reproducible development we run a single KRaft broker. Below is an excerpt from 'docker-compose.yml' that configures Kafka in KRaft mode (single-node):

```
1 kafka:
2   image: confluentinc/cp-kafka:7.6.0
3   container_name: kafka
4   ports:
5     - "9092:9092"
6     - "9093:9093"
7   environment:
8     KAFKA_NODE_ID: 1
9     KAFKA_PROCESS_ROLES: 'broker,controller'
10    KAFKA_CONTROLLER_QUORUM_VOTERS: '1@kafka:9093'
11    KAFKA_LISTENERS: 'PLAINTEXT://kafka:29092,PLAINTEXT_HOST://0.0.0.0:9092,CONTROLLER://kafka:9093'
12    KAFKA_ADVERTISED_LISTENERS: 'PLAINTEXT://kafka:29092,PLAINTEXT_HOST://localhost:9092'
13    KAFKA_LISTENER_SECURITY_PROTOCOL_MAP: 'CONTROLLER:PLAINTEXT,PLAINTEXT:PLAINTEXT,PLAINTEXT_HOST:PLAINTEXT'
14    KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1
15    KAFKA_AUTO_CREATE_TOPICS_ENABLE: 'true'
```

We also deploy a lightweight web UI for inspection and management:

```
1 kafka-ui:
2   image: provectuslabs/kafka-ui:latest
3   container_name: kafka-ui
4   ports:
5     - "8080:8080"
6   environment:
7     KAFKA_CLUSTERS_0_NAME: local
8     KAFKA_CLUSTERS_0_BOOTSTRAPSERVERS: kafka:29092
```

4.3.2 Debezium CDC Integration

Debezium Connect is configured to capture changes from the Postgres service and push CDC events to Kafka. The connector configuration is auto-registered at startup. The connector in this repo is named 'taxi-postgres-connector' and is configured with a 'topic.prefix' of 'taxi', capturing the 'public.taxi_trips' table; therefore the generated topic is 'taxi.public.taxi_trips'.

4.4 Minio

To serve as the storage backbone for our Data Lakehouse, we deploy MinIO, a high-performance, S3-compatible object storage server. MinIO provides the scalability and durability required for storing raw and processed data layers (Bronze, Silver, Gold) in the Iceberg format.

4.4.1 MinIO Cluster Configuration

Our MinIO deployment consists of a single server node and a client initialization container.

```
1 minio:
2   image: minio/minio:latest
3   container_name: minio
4   restart: always
5   environment:
6     MINIO_ROOT_USER: admin
7     MINIO_ROOT_PASSWORD: 12345678
8   ports:
9     - "9000:9000" # API
10    - "9001:9001" # Console
11   command: server /data --console-address ":9001"
12   volumes:
13     - minio_data:/data
```

To automate the provisioning of the storage buckets, we utilize a transient ‘mc’ (MinIO Client) container. This service waits for the MinIO server to become available, then creates the primary ‘lakehouse’ bucket and configures its access policies.

```
1 mc:
2   depends_on:
3     - minio
4   image: minio/mc
5   container_name: mc
6   environment:
7     - AWS_ACCESS_KEY_ID=admin
8     - AWS_SECRET_ACCESS_KEY=12345678
9     - AWS_REGION=us-east-1
10  entrypoint: >
11    /bin/sh -c "
12    until (/usr/bin/mc alias set minio http://minio:9000 admin 12345678) do echo '...waiting...'
13    && sleep 1; done;
14    /usr/bin/mc rm -r --force minio/lakehouse;
15    /usr/bin/mc mb minio/lakehouse;
16    /usr/bin/mc policy set public minio/lakehouse;
17    tail -f /dev/null
18  "
```

4.4.2 Spark Integration

The integration between Apache Spark and MinIO is achieved through the Hadoop S3A file system client. This allows Spark to interact with MinIO as if it were Amazon S3. The configuration below demonstrates how the Spark session is initialized with the necessary S3A and Iceberg settings to connect to the ‘lakehouse’ catalog.

```
1 def get_spark():
2   s3a_endpoint = os.getenv("S3A_ENDPOINT", "http://localhost:9000")
3   s3a_access_key = os.getenv("S3A_ACCESS_KEY", "admin")
4   s3a_secret_key = os.getenv("S3A_SECRET_KEY", "12345678")
5   iceberg_warehouse = os.getenv("ICEBERG_WAREHOUSE", "s3a://lakehouse/warehouse")
6
7   return (
8     SparkSession.builder
9     .appName("TaxiStreamToIceberg")
10    .config(
11      "spark.jars.packages",
12      ",".join([
```

```

13         "org.apache.spark:spark-sql-kafka-0-10_2.12:3.5.1",
14         "org.apache.iceberg:iceberg-spark-runtime-3.5_2.12:1.6.0",
15         "org.apache.hadoop:hadoop-aws:3.3.4",
16         "com.amazonaws:aws-java-sdk-bundle:1.12.767",
17     ])
18 )
19 # Iceberg Catalog Configuration
20 .config("spark.sql.catalog.lakehouse", "org.apache.iceberg.spark.SparkCatalog")
21 .config("spark.sql.catalog.lakehouse.type", "hadoop")
22 .config("spark.sql.catalog.lakehouse.warehouse", iceberg_warehouse)
23
24 # S3A Configuration for MinIO
25 .config("spark.hadoop.fs.s3a.endpoint", s3a_endpoint)
26 .config("spark.hadoop.fs.s3a.access.key", s3a_access_key)
27 .config("spark.hadoop.fs.s3a.secret.key", s3a_secret_key)
28 .config("spark.hadoop.fs.s3a.path.style.access", "true")
29 .config("spark.hadoop.fs.s3a.impl", "org.apache.hadoop.fs.s3a.S3AFileSystem")
30 .config("spark.hadoop.fs.s3a.connection.ssl.enabled", "false")
31 .getOrCreate()
32 )

```

4.5 Spark

4.5.1 Spark Thrift Server (JDBC/ODBC Gateway)

The Spark Thrift Server provides a JDBC/ODBC interface that allows external BI tools (such as Streamlit, Superset, and Tableau) to execute SQL queries against the Lakehouse. It acts as a bridge between the Hive Metastore and visualization clients, enabling them to query Iceberg tables stored in MinIO using standard SQL.

4.5.2 Spark Thrift Server Configuration

The Spark Thrift Server is deployed as a standalone container that connects to the Hive Metastore for table metadata and MinIO (S3-compatible storage) for data access:

```

1 spark-thrift:
2   container_name: spark-thrift
3   image: spark:3.5.6-scala2.12-java17-python3-ubuntu
4   entrypoint: ['./entrypoint.sh', 'thrift']
5   volumes:
6     - ./spark/entrypoint.sh:/opt/spark/work-dir/entrypoint.sh
7     - ./spark/spark-defaults.conf:/opt/spark/conf/spark-defaults.conf
8     - ./data:/opt/spark/data
9     - spark-logs:/opt/spark/spark-events
10  ports:
11    - 10000:10000 # HiveServer2 Thrift Protocol (JDBC)
12  depends_on:
13    - hive-metastore
14    - minio

```

Key configuration aspects:

- **Port 10000:** The Thrift server exposes the standard HiveServer2 port, compatible with any JDBC client that supports Hive.
- **Volumes:** The entrypoint script and Spark configuration are mounted to customize startup behavior and storage connectivity.

- **Dependencies:** The server waits for the Hive Metastore and MinIO to be available before starting.

4.5.3 Entrypoint and Dependency Management

The entrypoint script (entrypoint.sh) dynamically downloads required JARs at runtime and starts the Thrift server with the appropriate configuration:

```

1 #!/bin/bash
2 SPARK_WORKLOAD=$1
3
4 echo "Downloading Hadoop AWS and AWS SDK JARs..."
5
6 cd /opt/spark/jars
7 wget -q "https://repo1.maven.org/.../hadoop-aws-3.3.4.jar"
8 wget -q "https://repo1.maven.org/.../aws-java-sdk-bundle-1.12.262.jar"
9 wget -q "https://repo1.maven.org/.../delta-spark_2.12-3.3.1.jar"
10 wget -q "https://repo1.maven.org/.../delta-storage-3.3.1.jar"
11 wget -q "https://jdbc.postgresql.org/.../postgresql-42.6.0.jar"
12 wget -q "https://repo1.maven.org/.../delta-hive_2.12-3.3.1.jar"
13
14 if [ "$SPARK_WORKLOAD" == "thrift" ]; then
15     sleep 10 # Wait for metastore initialization
16     /bin/bash /opt/spark/sbin/start-thriftserver.sh \
17         --jars /opt/spark/jars/hadoop-aws-3.3.4.jar,\
18             /opt/spark/jars/aws-java-sdk-bundle-1.12.262.jar,\
19             /opt/spark/jars/delta-spark_2.12-3.3.1.jar,\
20             /opt/spark/jars/delta-storage-3.3.1.jar,\
21             /opt/spark/jars/postgresql-42.6.0.jar,\
22             /opt/spark/jars/delta-hive_2.12-3.3.1.jar \
23         --hiveconf hive.metastore.uris=thrift://hive-metastore:9083
24 fi
25
26 tail -f /dev/null # Keep container alive

```

The downloaded dependencies enable:

- **hadoop-aws + aws-java-sdk-bundle:** S3A filesystem support for reading/writing data to MinIO.
- **delta-spark + delta-storage:** Delta Lake table format support for ACID transactions.
- **delta-hive:** Enables the Hive Metastore to understand Delta Lake table metadata.
- **postgresql:** JDBC driver for potential PostgreSQL connectivity (used by other components).

4.5.4 Spark Storage Configuration

The spark-defaults.conf file configures MinIO as the S3-compatible storage backend and enables Delta Lake extensions:

```

1 # MinIO Configuration (S3A FileSystem)
2 spark.hadoop.fs.s3a.endpoint=http://minio:9000
3 spark.hadoop.fs.s3a.access.key=admin
4 spark.hadoop.fs.s3a.secret.key=12345678
5 spark.hadoop.fs.s3a.path.style.access=true
6 spark.hadoop.fs.s3a.impl=org.apache.hadoop.fs.s3a.S3AFileSystem
7
8 # Delta Lake Extensions
9 spark.sql.extensions=io.delta.sql.DeltaSparkSessionExtension
10 spark.sql.catalog.spark_catalog=org.apache.spark.sql.delta.catalog.DeltaCatalog

```

This configuration enables:

- **Path-style access:** Required for MinIO since it does not support virtual-hosted-style URLs.
- **Delta Session Extension:** Enables Delta Lake SQL commands (MERGE, DELETE, UPDATE).
- **Delta Catalog:** Registers the Delta catalog for seamless table management.

4.5.5 Spark Medallion Processing

Bronze — ingest and preserve

The Bronze layer ingests raw Kafka/Debezium events with minimal transformation and preserves original payload fields (typically as strings or their raw types). Bronze includes ingestion metadata (timestamps, kafka offsets) used for latency tracing and troubleshooting. The Bronze job runs as a streaming job (checkpointed foreachBatch writes) and is intentionally append-oriented to keep a faithful event trail.

```

1      -- Example Bronze table (Iceberg)
2      CREATE TABLE IF NOT EXISTS lakehouse.bronze.taxi_trips (
3          id INTEGER,
4          tpep_pickup_datetime STRING,
5          tpep_dropoff_datetime STRING,
6          passenger_count STRING,
7          trip_distance STRING,
8          ...
9      ) USING iceberg;
10

```

Silver — clean, normalize, and enrich

Silver reads Bronze data and performs deterministic cleansing and typing: converts timestamp strings to `TIMESTAMP`, casts numerics, applies small UDF mappings (e.g., payment code → label), computes derived metrics (trip duration), filters invalid rows, fills/defaults missing numeric columns, and deduplicates. The Silver job runs incrementally using a lightweight ‘watermarks’ Iceberg table that records ‘last_watermark_t’(epochms) so as subsequent run only processes newer records.

```

1      -- Watermark table (sample)
2      CREATE TABLE IF NOT EXISTS lakehouse.bronze.watermarks (
3          table_name STRING,
4          last_watermark_ts LONG,
5          updated_at TIMESTAMP
6      ) USING iceberg;
7
8      -- Example incremental filter used by Silver/Gold
9      SELECT * FROM lakehouse.bronze.taxi_trips
10     WHERE CAST(tpep_pickup_datetime AS LONG) > '<last_watermark_ts>';
11

```

Gold — dimensional modeling and aggregates

Gold consumes incremental Silver data (watermark-driven) and produces curated assets for analytics:

- Dimensions (e.g., ‘dim_datetime’, ‘dim_zone’): canonical surrogate keys (e.g., ‘datetime_sk’) are generated and maintained using MERGE/upsert semantics so dimension tables remain compact and consistent.

- Fact table ('fact_trip'): denormalized fact with foreign keys to dimensions (e.g., 'datetime_sk', 'zone_id'), typed numeric columns, and business-friendly fields (fare, tip, trip duration, payment label).

Major operations at Gold use Iceberg MERGE statements to perform idempotent upserts and avoid duplicates:

```

1  MERGE INTO lakehouse.gold.fact_trip AS target
2  USING fact_trip_temp AS source
3  ON target.pickup_datetime_sk = source.pickup_datetime_sk
4  AND target.dropoff_datetime_sk = source.dropoff_datetime_sk
5  AND target.pickup_zone_id = source.pickup_zone_id
6  AND target.dropoff_zone_id = source.dropoff_zone_id
7  WHEN NOT MATCHED THEN INSERT *;
8

```

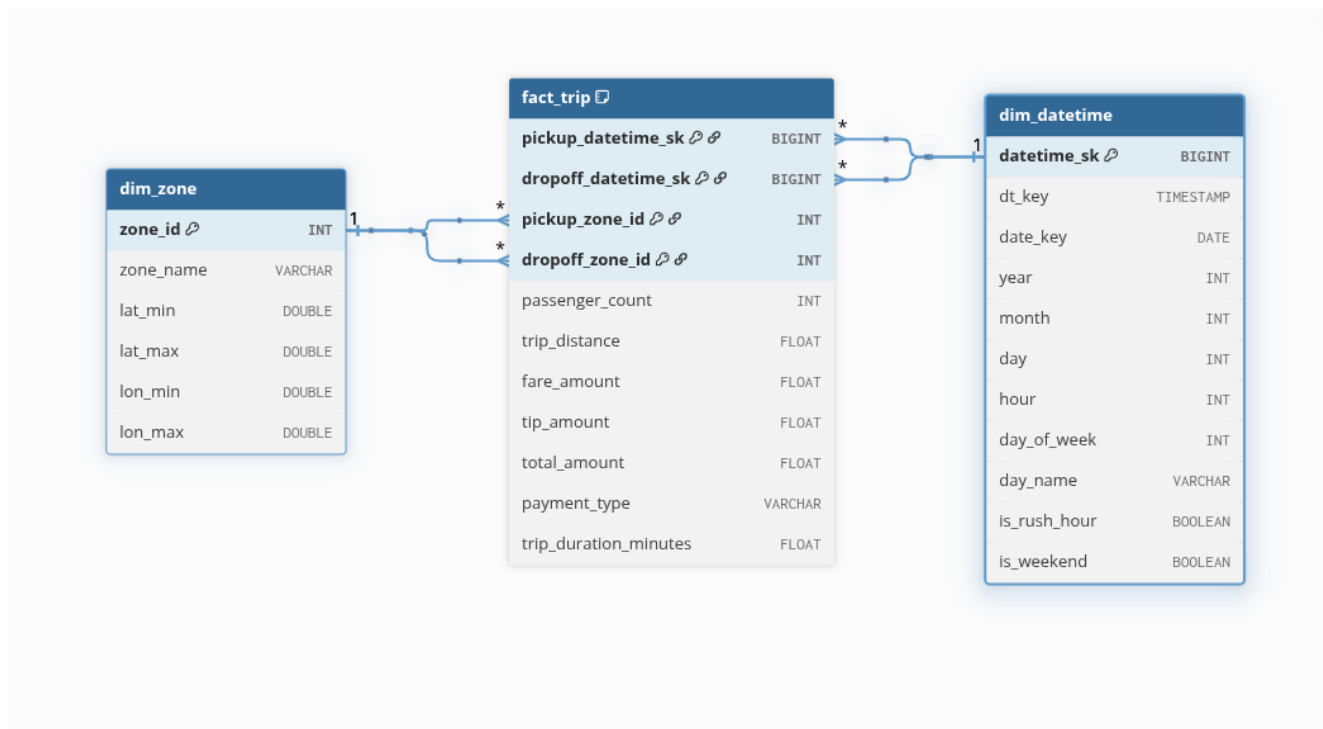


Figure 2: Data model at gold layer

```

1  -- Dimension Table: DateTime
2  CREATE TABLE IF NOT EXISTS lakehouse.gold.dim_datetime (
3      datetime_sk BIGINT,
4      dt_key TIMESTAMP,
5      date_key DATE,
6      year INT, month INT, day INT, hour INT,
7      day_of_week INT,
8      day_name STRING,
9      is_rush_hour BOOLEAN,
10     is_weekend BOOLEAN
11 ) USING iceberg;
12
13 -- Dimension Table: Zone
14 CREATE TABLE IF NOT EXISTS lakehouse.gold.dim_zone (
15     zone_id INT,
16     zone_name STRING,
17     lat_min DOUBLE, lat_max DOUBLE,
18     lon_min DOUBLE, lon_max DOUBLE

```

```

19 ) USING iceberg;
20
21 -- Fact Table: Trip
22 CREATE TABLE IF NOT EXISTS lakehouse.gold.fact_trip (
23     pickup_datetime_sk BIGINT,
24     dropoff_datetime_sk BIGINT,
25     pickup_zone_id INT,
26     dropoff_zone_id INT,
27     passenger_count INT,
28     trip_distance FLOAT,
29     fare_amount FLOAT,
30     tip_amount FLOAT,
31     total_amount FLOAT,
32     payment_type STRING,
33     trip_duration_minutes FLOAT
34 ) USING iceberg;

```

4.6 Flink

In our platform, Apache Flink is responsible for the **low-latency analytics layer** that operates on the continuous stream of taxi trip events. While Kafka (fed by Debezium CDC) provides the durable event log, Flink performs stateful, event-time windowed aggregation and persists the real-time KPIs into Cassandra for fast dashboard queries.

The cluster consists of a **JobManager** (control plane) and at least one **TaskManager** (data plane). The JobManager exposes the Flink Web UI, allowing us to inspect job status, subtasks, checkpoints, and backpressure information.

4.6.1 Flink Cluster Configuration

The Flink runtime is defined in `docker-compose.yml`. The JobManager coordinates scheduling and checkpointing, while TaskManagers execute the streaming operators. In our configuration we enable a small parallel runtime (2 slots) suitable for local development:

The Flink image used by this repository is pre-loaded with the connectors required for our pipeline. Specifically, the custom Flink image adds the Kafka and Cassandra connectors into `/opt/flink/lib/`, letting a submitted job communicate with Kafka and Cassandra without additional classpath configuration:

```

1 FROM flink:1.19
2
3 # Kafka connector
4 ADD https://repo1.maven.org/maven2/org/apache/flink/flink-connector-kafka/3.3.0-1.19/flink-connector-kafka-3.3.0-1.19.jar /opt/flink/lib/
5 ADD https://repo1.maven.org/maven2/org/apache/kafka/kafka-clients/3.7.0/kafka-clients-3.7.0.jar /opt/flink/lib/
6
7 # Cassandra connector
8 ADD https://repo1.maven.org/maven2/org/apache/flink/flink-connector-cassandra_2.12/3.2.0-1.19/flink-connector-cassandra_2.12-3.2.0-1.19.jar /opt/flink/lib/

```

4.6.2 Streaming Job: Kafka (CDC) → Flink (KPIs) → Cassandra

Our Flink job is implemented in Java (module `src/flink/app`) with entry point `org.streaming.Application`. The job consumes taxi trip change events from Kafka, assigns event-time watermarks (bounded out-

of-orderness), and computes two real-time metrics:

- **Revenue per window:** a tumbling event-time window (10 seconds in the current implementation) aggregates total revenue and number of trips.
- **Active trips:** a sliding event-time window (size 2 minutes, slide 30 seconds) counts the number of trips observed in the recent interval.

Both outputs are written to Cassandra (keyspace `realtime`), enabling dashboard-style queries with low latency.

The core job topology is summarized below (simplified for readability):

```
1 KafkaSource<String> source = KafkaSource.<String>builder()
2   .setBootstrapServers("kafka:29092")
3   .setTopics("taxi.public.taxi_trips")
4   .setGroupId("flink-taxi-analytics")
5   .setStartingOffsets(OffsetsInitializer.earliest())
6   .setValueOnlyDeserializer(new SimpleStringSchema())
7   .build();
8
9 DataStream<TaxiTrip> trips = env.fromSource(source, WatermarkStrategy.noWatermarks(), "Kafka
   Source")
10  .map(CDCParser::parseDebeziumAfter)
11  .filter(t -> t != null)
12  .assignTimestampsAndWatermarks(
13      WatermarkStrategy.<TaxiTrip>forBoundedOutOfOrderness(Duration.ofSeconds(10))
14      .withTimestampAssigner((trip, ts) -> trip.pickupTime)
15  );
16
17 DataStream<RevenuePerMinute> revenue = trips
18  .keyBy(t -> 1)
19  .window(TumblingEventTimeWindows.of(Duration.ofSeconds(10)))
20  .aggregate(new RevenueAggregator(), new RevenueWindowFunction());
21
22 CassandraSink.addSink(revenue)
23  .setHost("cassandra", 9042)
24  .build();
```

Finally, the job is executed and monitored via the Flink Web UI, where we can validate operator parallelism, throughput, and end-to-end lag. When paired with the Cassandra tables defined in the next section, this completes the real-time analytics loop from CDC events to queryable KPIs.

4.7 Cassandra

Our Cassandra instance is deployed as a Docker container for reproducibility and local testing. We use a single-node Cassandra with the default CQL port exposed for clients:

```
1 cassandra:
2 image: cassandra:4.1
3 container_name: cassandra
4 restart: always
5 ports:
6 - "9042:9042" # CQL
7 environment:
8 CASSANDRA_CLUSTER_NAME: "TaxiCluster"
9 CASSANDRA_NUM_TOKENS: 256
10 CASSANDRA_START_RPC: "true"
11 CASSANDRA_RPC_ADDRESS: 0.0.0.0
12 CASSANDRA_BROADCAST_RPC_ADDRESS: cassandra
13 CASSANDRA_BROADCAST_ADDRESS: cassandra
14 MAX_HEAP_SIZE: "1G"
```

```

15 HEAP_NEWSIZE: "256M"
16 volumes:
17 - cassandra_data:/var/lib/cassandra
18 healthcheck:
19 test: ["CMD-SHELL", "cqlsh -e 'DESCRIBE KEYSPACES' || exit 1"]

```

```

1 flink-jobmanager:
2   image: ducngoquang/flink:1.19
3   container_name: flink-jobmanager
4   restart: always
5   ports:
6     - "8081:8081"    # Flink UI
7   command: jobmanager
8   environment:
9     - |
10      FLINK_PROPERTIES=
11      jobmanager.rpc.address: flink-jobmanager
12   networks:
13     - taxi-platform
14
15 flink-taskmanager:
16   image: ducngoquang/flink:1.19
17   container_name: flink-taskmanager
18   restart: always
19   depends_on:
20     - flink-jobmanager
21   command: taskmanager
22   environment:
23     - |
24      FLINK_PROPERTIES=
25      jobmanager.rpc.address: flink-jobmanager
26      taskmanager.numberOfTaskSlots: 2
27   networks:
28     - taxi-platform

```

4.7.1 Schema and Tables

We create a keyspace and two tables used by the Flink streaming jobs to store aggregated metrics (minute-level revenue and active trips):

```

1
2 CREATE KEYSPACE IF NOT EXISTS realtime
3 WITH replication = {'class': 'SimpleStrategy', 'replication_factor': 1};
4
5 CREATE TABLE IF NOT EXISTS realtime.revenue_minute (
6   window_start timestamp PRIMARY KEY,
7   window_end timestamp,
8   total_revenue double,
9   trip_count bigint
10 );
11
12 CREATE TABLE IF NOT EXISTS realtime.active_trips (
13   window_start timestamp PRIMARY KEY,
14   window_end timestamp,
15   active_trips bigint
16 );

```

4.8 Airflow

Airflow serves as the orchestration backbone of our data platform, responsible for scheduling and monitoring the execution of our Spark-based ETL pipeline. It ensures that data transformations occur reliably and on schedule, with proper dependency management and retry logic.

4.8.1 Airflow Deployment Architecture

Our Airflow deployment consists of two main Docker services: a dedicated PostgreSQL database for metadata storage and the Airflow application itself running both the webserver and scheduler components.

The metadata database is configured as a lightweight PostgreSQL instance:

```
1 airflow-db:
2   image: postgres:16-alpine
3   container_name: airflow-db
4   restart: always
5   environment:
6     POSTGRES_USER: airflow
7     POSTGRES_PASSWORD: airflow
8     POSTGRES_DB: airflow
9   volumes:
10    - ./airflow/db_data:/var/lib/postgresql/data
11   networks:
12    - taxi-platform
```

The Airflow service is built from a custom Docker image that extends the official Apache Airflow 2.10.0 base image with Java 17 support, enabling seamless integration with Spark jobs:

```
1 FROM apache/airflow:2.10.0-python3.10
2
3 USER root
4
5 # Install Java 17 and procs
6 RUN apt-get update && apt-get install -y openjdk-17-jdk procps && \
7     apt-get clean && rm -rf /var/lib/apt/lists/*
8
9 # Set JAVA_HOME
10 ENV JAVA_HOME=/usr/lib/jvm/java-17-openjdk-amd64
11 ENV PATH=$JAVA_HOME/bin:$PATH
12
13 USER airflow
14
15 # Copy requirements and install Python packages during build
16 COPY airflow-requirements.txt /tmp/airflow-requirements.txt
17 RUN pip install -r /tmp/airflow-requirements.txt
```

The Airflow container is configured with the LocalExecutor for task execution and mounted volumes for DAG definitions, logs, and source code access:

```
1 airflow:
2   build:
3     context: ./src/airflow
4     dockerfile: Dockerfile.airflow
5   image: airflow-java:2.10.0
6   container_name: airflow
7   hostname: airflow
8   restart: always
9   depends_on:
10    - airflow-db
```

```

11 environment:
12     AIRFLOW__DATABASE__SQL_ALCHEMY_CONN: postgresql+psycopg2://airflow:airflow@airflow-db:5432/
    airflow
13     AIRFLOW__CORE__LOAD_EXAMPLES: "False"
14     AIRFLOW__DATABASE__LOAD_DEFAULT_CONNECTIONS: "False"
15     AIRFLOW__CORE__DAGS_FOLDER: /opt/airflow/dags
16     AIRFLOW__CORE__EXECUTOR: LocalExecutor
17     AIRFLOW__LOGGING__BASE_LOG_FOLDER: /opt/airflow/logs
18     AIRFLOW__LOGGING__LOGGING_LEVEL: INFO
19 ports:
20     - "8085:8080"
21 volumes:
22     - ./src/airflow/dags:/opt/airflow/dags
23     - ./src:/opt/airflow/src
24     - ./airflow/logs:/opt/airflow/logs
25 command: >
26     bash -c "
27         airflow db upgrade &&
28         airflow db init &&
29         airflow users create --username admin --password admin
30             --firstname Admin --lastname User --role Admin
31             --email admin@example.com || true;
32         airflow webserver -p 8080 &
33         airflow scheduler
34     "
35 networks:
36     - taxi-platform

```

The startup command performs the following initialization steps:

1. **Database Migration:** Upgrades the metadata database schema to match the current Airflow version.
2. **Admin User Creation:** Creates a default admin user (username: admin, password: admin) for web UI access.
3. **Dual Process Launch:** Starts both the webserver (port 8080) and scheduler as background processes within the same container.

4.8.2 Spark Connection Configuration

To enable Airflow to submit jobs to our standalone Spark cluster, we programmatically create a Spark connection using the Airflow Python API:

```

1 from airflow import settings
2 from airflow.models import Connection
3
4 # Delete old connection if exists
5 session = settings.Session()
6 conn = session.query(Connection).filter(
7     Connection.conn_id == 'spark_default'
8 ).first()
9 if conn:
10     session.delete(conn)
11     session.commit()
12
13 # Create new connection pointing to Spark master
14 new_conn = Connection(
15     conn_id='spark_default',
16     conn_type='spark',
17     host='spark://spark-master',
18     port=7077,

```



```

19     extra={'deploy-mode': 'client'},
20 )
21
22 session.add(new_conn)
23 session.commit()
24 session.close()

```

This connection configuration specifies:

- **Connection ID:** `spark_default` – referenced by `SparkSubmitOperator` tasks
- **Master URL:** `spark://spark-master:7077` – points to the Spark master service
- **Deploy Mode:** `client` – runs the driver inside the Airflow container for simplified debugging

4.8.3 Lakehouse Pipeline DAG

Our primary workflow is the `spark_lakehouse_pipeline` DAG, which orchestrates the daily execution of our medallion architecture transformations. The DAG is scheduled to run daily and consists of two sequential tasks: Silver layer processing followed by Gold layer aggregation.

```

1 from airflow import DAG
2 from airflow.providers.apache.spark.operators.spark_submit import SparkSubmitOperator
3 from datetime import datetime, timedelta
4
5 default_args = {
6     'owner': 'airflow',
7     'depends_on_past': False,
8     'email_on_failure': False,
9     'retries': 1,
10    'retry_delay': timedelta(minutes=5),
11 }
12
13 with DAG(
14     dag_id='spark_lakehouse_pipeline',
15     default_args=default_args,
16     description='Run Spark Silver and Gold jobs daily',
17     schedule='0 2 * * *',
18     start_date=datetime(2025, 11, 1),
19     catchup=False,
20     tags=['spark', 'lakehouse', 'silver', 'gold'],
21 ) as dag:
22
23     run_silver = SparkSubmitOperator(
24         task_id='run_silver_job',
25         application='/opt/airflow/src/silver.py',
26         conn_id='spark_default',
27         deploy_mode='client',
28         spark_binary='spark-submit',
29         executor_cores=2,
30         executor_memory='1g',
31         driver_memory='512m',
32         total_executor_cores=2,
33         verbose=True,
34         name='silver-job',
35         packages='org.apache.spark:spark-sql-kafka-0-10_2.12:3.5.1,org.apache.iceberg:iceberg-
spark-runtime-3.5_2.12:1.6.0,org.apache.hadoop:hadoop-aws:3.3.4,com.amazonaws:aws-java-sdk-
bundle:1.12.767',
36         conf={
37             'spark.pyspark.python': 'python3',
38             'spark.pyspark.driver.python': 'python3',
39         },
40     )

```

```

41
42 run_gold = SparkSubmitOperator(
43     task_id='run_gold_job',
44     application='/opt/airflow/src/gold.py',
45     conn_id='spark_default',
46     deploy_mode='client',
47     spark_binary='spark-submit',
48     executor_cores=2,
49     executor_memory='1g',
50     driver_memory='512m',
51     total_executor_cores=2,
52     verbose=True,
53     name='gold-job',
54     packages='org.apache.spark:spark-sql-kafka-0-10_2.12:3.5.1,org.apache.iceberg:iceberg-
spark-runtime-3.5_2.12:1.6.0,org.apache.hadoop:hadoop-aws:3.3.4,com.amazonaws:aws-java-sdk-
bundle:1.12.767',
55     conf={
56         'spark.pyspark.python': 'python3',
57         'spark.pyspark.driver.python': 'python3',
58     },
59 )
60
61 # Define task dependency
62 run_silver >> run_gold

```

Each SparkSubmitOperator is configured with:

- **Resource Allocation:** 2 executor cores with 1GB memory per executor and 512MB driver memory, suitable for our development workload.
- **Maven Packages:** Runtime dependencies including Spark-Kafka integration, Iceberg support, and AWS S3 connectivity for MinIO storage.
- **Python Environment:** Explicit Python 3 configuration ensures compatibility between the Airflow and Spark Python environments.
- **Task Dependency:** The Silver job must complete successfully before the Gold job begins, enforcing proper data flow through the medallion layers.

4.8.4 Monitoring and Observability

The Airflow web UI (accessible at <http://localhost:8085>) provides comprehensive monitoring capabilities:

- **DAG View:** Visualizes task dependencies and execution status for each DAG run.
- **Task Logs:** Real-time streaming logs from Spark jobs, including stdout, stderr, and Spark driver output.
- **Gantt Chart:** Timeline visualization of task execution showing bottlenecks and parallelization opportunities.
- **Retry Mechanism:** Automatic retry with 5-minute delay upon task failure, providing resilience against transient errors.

All task logs are persisted to the `./airflow/logs` directory, facilitating debugging and audit trail maintenance.

The successful completion of these tasks also implicitly verifies our infrastructure configuration, proving that the Airflow container has valid access to the host's Docker socket and can successfully submit jobs to the spark-master container.

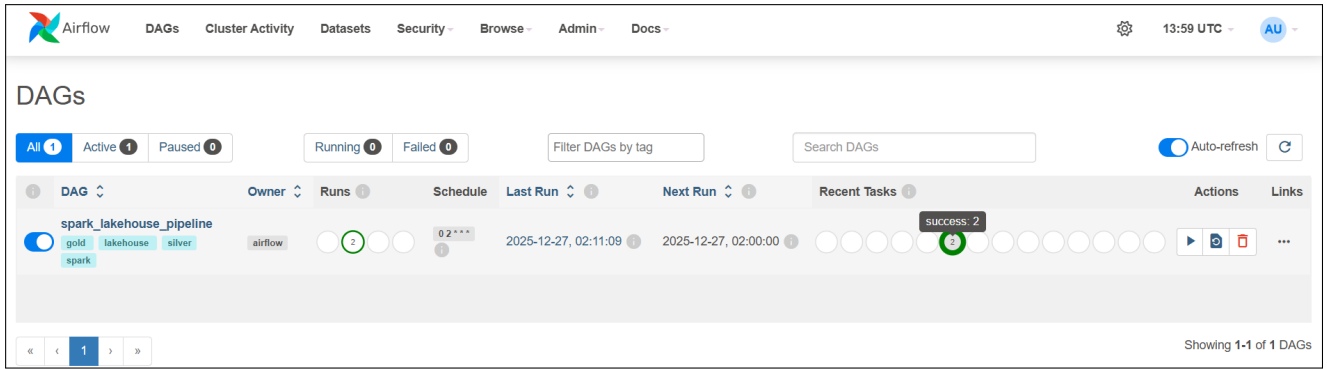


Figure 3: Airflow Dashboard: Verification of active DAGs, schedules, and successful run history.

4.9 Streamlit

4.10 Data Visualization with Streamlit and PyHive

To provide business users with interactive insights into the NYC taxi analytics data, we developed a custom dashboard using **Streamlit** and **PyHive**. This visualization layer connects directly to the Spark Thrift Server, enabling real-time SQL queries against the Gold layer Iceberg tables stored in MinIO.

4.10.1 Dashboard Containerization

The Streamlit dashboard is deployed as a containerized application with all dependencies pre-installed:

```
1 FROM python:3.11-slim
2
3 WORKDIR /app
4
5 # Install system dependencies for PyHive
6 RUN apt-get update && apt-get install -y \
7     gcc \
8     libsasl2-dev \
9     libsasl2-modules \
10    && rm -rf /var/lib/apt/lists/*
11
12 COPY requirements.txt .
13 RUN pip install --no-cache-dir -r requirements.txt
14
15 COPY . .
16
17 EXPOSE 8501
18
19 CMD ["streamlit", "run", "dashboard.py", "--server.address=0.0.0.0"]
```

The dashboard container depends on the Spark Thrift Server and is configured via environment variables:

```
1 streamlit-dashboard:
2   build:
3     context: ./streamlit
4     dockerfile: Dockerfile
5   container_name: streamlit-dashboard
6   restart: always
7   ports:
```

```

8     - "8501:8501"
9     depends_on:
10         - spark-thrift
11     environment:
12         - SPARK_HOST=spark-thrift
13         - SPARK_PORT=10000

```

4.10.2 PyHive Connection to Spark Thrift Server

PyHive provides a Python DB-API interface to HiveServer2-compatible endpoints. We use it to establish connections to the Spark Thrift Server and execute SQL queries against the Gold layer:

```

1 from pyhive import hive
2 from contextlib import contextmanager
3 import pandas as pd
4
5 # Configuration
6 SPARK_HOST = "spark-thrift"
7 SPARK_PORT = 10000
8 DATABASE = "lakehouse.gold"
9
10 @contextmanager
11 def get_connection():
12     """Create a connection to Spark Thrift Server"""
13     conn = None
14     try:
15         conn = hive.Connection(
16             host=SPARK_HOST,
17             port=SPARK_PORT,
18             username="hive"
19         )
20         yield conn
21     finally:
22         if conn:
23             conn.close()
24
25 @st.cache_data(ttl=300)
26 def run_query(query: str) -> pd.DataFrame:
27     """Execute SQL query and return DataFrame"""
28     try:
29         with get_connection() as conn:
30             return pd.read_sql(query, conn)
31     except Exception as e:
32         st.error(f"Query failed: {e}")
33     return pd.DataFrame()

```

Key aspects of the connection layer:

- **Context Manager:** Ensures proper connection cleanup after each query.
- **TTL Caching:** Streamlit's `cache_data` decorator reduces redundant queries by caching results for 5 minutes.
- **Pandas Integration:** Query results are returned as DataFrames for easy manipulation and visualization.

4.10.3 Python Dependencies

The dashboard requires the following packages to connect to Spark Thrift Server via PyHive:

```

1 streamlit==1.40.0
2 plotly==5.24.0

```

```

3 pandas==2.2.0
4 pyhive==0.7.0
5 thrift==0.20.0
6 pure-sasl==0.6.2
7 thrift_sasl==0.4.3

```

Key dependencies:

- **pyhive:** Python DB-API interface for HiveServer2.
- **thrift & thrift_sasl:** Thrift protocol libraries for HiveServer2 communication.
- **plotly:** Interactive charting library for data visualization.

4.10.4 Business Intelligence Dashboard

The dashboard is organized into four interconnected business-story sections, providing stakeholders with actionable insights:

Executive Overview

High-level KPIs including total revenue, trip count, average revenue per trip, and average trip duration. The section also compares rush hour vs. off-peak performance:

```

1 def get_executive_kpis() -> dict:
2     """Fetch executive overview KPIs"""
3     query = """
4     SELECT
5         SUM(total_amount) as total_revenue,
6         COUNT(*) as total_trips,
7         AVG(total_amount) as avg_revenue_per_trip,
8         AVG(fare_amount) as avg_fare,
9         AVG(tip_amount) as avg_tip,
10        SUM(passenger_count) as total_passengers,
11        AVG(trip_distance) as avg_distance,
12        AVG(trip_duration_minutes) as avg_duration
13    FROM gold_fact_trip
14    """
15    df = run_query(query)
16    if not df.empty:
17        return df.iloc[0].to_dict()
18    return {}

```

Figure 4 illustrates this executive layer, where stakeholders can immediately assess the health of daily operations. The KPI cards provide instant visibility into total revenue generation, while the rush hour comparison reveals pricing opportunities during peak demand periods.

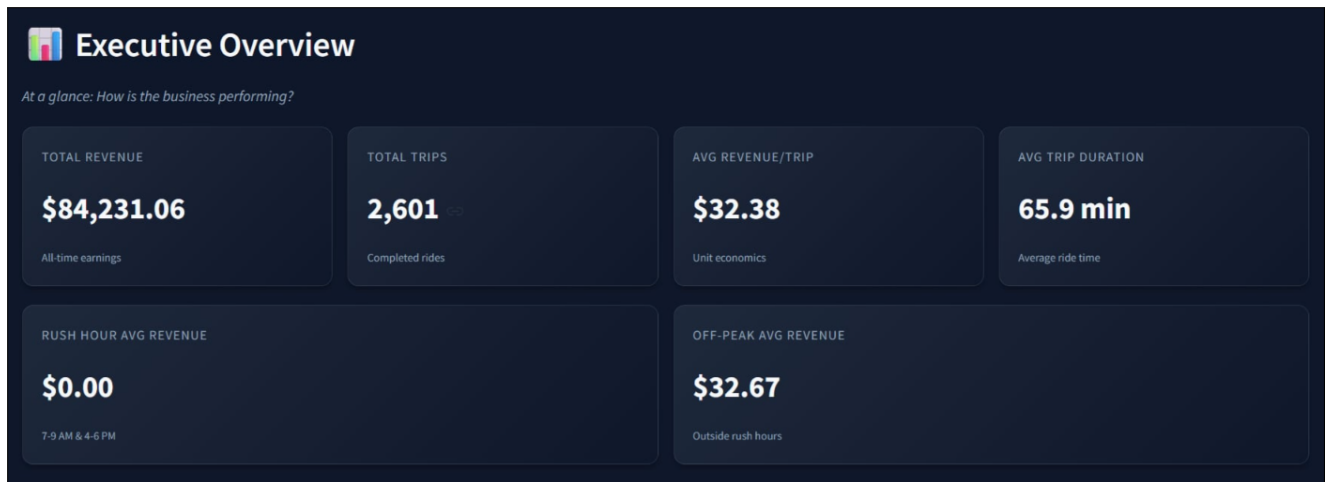


Figure 4: Executive Overview: Real-time KPIs displaying total revenue (\$39,932.03), trip count (2,389), and average metrics. The rush hour comparison reveals higher per-trip revenue during peak hours.

Revenue Deep-Dive

Analyzes revenue sources through payment type breakdown (donut chart), hourly revenue patterns (bar chart), and top revenue-generating routes:

```

1 def get_top_routes() -> pd.DataFrame:
2     """Top revenue routes with zone names"""
3     query = """
4     SELECT
5         pz.zone_name as pickup_zone,
6         dz.zone_name as dropoff_zone,
7         SUM(f.total_amount) as revenue,
8         COUNT(*) as trips,
9         AVG(f.total_amount) as avg_fare
10    FROM gold_fact_trip f
11    JOIN gold_dim_zone pz ON f.pickup_zone_id = pz.zone_id
12    JOIN gold_dim_zone dz ON f.dropoff_zone_id = dz.zone_id
13    GROUP BY pz.zone_name, dz.zone_name
14    ORDER BY revenue DESC
15    LIMIT 10
16    """
17    return run_query(query)

```

This revenue analysis layer (Figure 5) enables finance teams to understand which routes contribute most to the bottom line. The horizontal bar chart ranks routes by total revenue, revealing that intra-zone trips within Manhattan and Brooklyn dominate the revenue landscape.

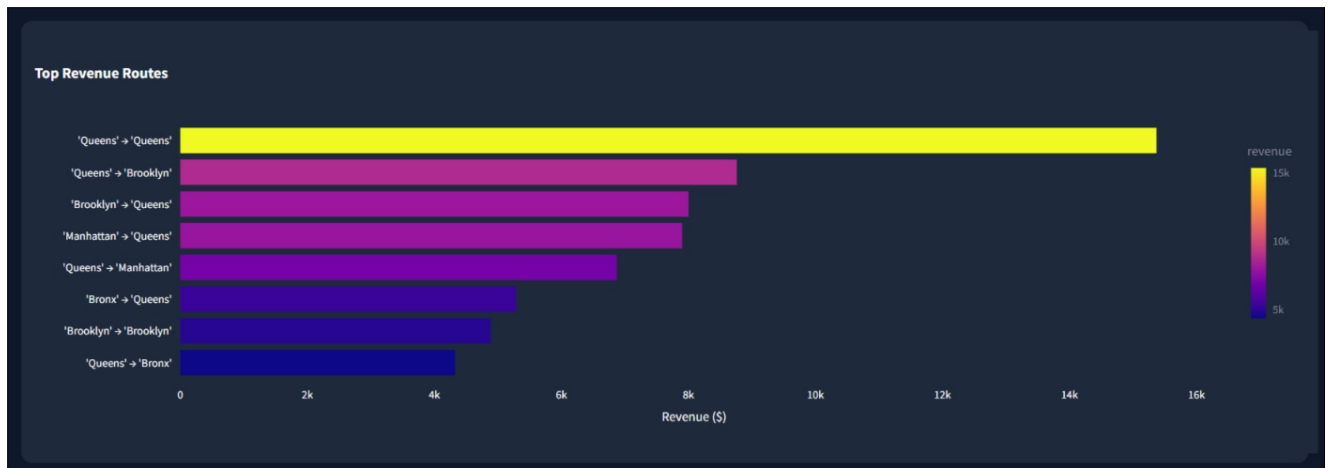


Figure 5: Revenue Deep-Dive: Top revenue-generating routes ranked by total earnings. Manhattan-to-Manhattan and Brooklyn-to-Brooklyn routes generate the highest revenue, informing fleet allocation strategies.

Operations & Efficiency

Visualizes operational metrics including average tip by payment type and trip duration distribution (bucketed histogram). As shown in Figure 6, credit card payments yield significantly higher tips compared to other payment methods, while the trip duration histogram reveals that most rides fall within the 10-30 minute range—guiding driver scheduling and route optimization.



Figure 6: Operations & Efficiency: Average tip analysis by payment type shows credit cards yield \$2.00+ tips, while the duration distribution confirms the majority of trips complete within 10-30 minutes.

Geographic Intelligence

Provides spatial insights through a zone-to-zone trip flow heatmap and revenue breakdown by pickup zone:

```

1 def get_zone_heatmap() -> pd.DataFrame:
2     """Zone-to-zone trip flow for heatmap"""
3     query = """
4     SELECT
5         pz.zone_name as pickup_zone,
6         dz.zone_name as dropoff_zone,
7         COUNT(*) as trips,

```

```

8         SUM(f.total_amount) as revenue
9     FROM gold_fact_trip f
10    JOIN gold_dim_zone pz ON f.pickup_zone_id = pz.zone_id
11    JOIN gold_dim_zone dz ON f.dropoff_zone_id = dz.zone_id
12    GROUP BY pz.zone_name, dz.zone_name
13    """
14    return run_query(query)

```

The geographic layer (Figure 7) provides spatial decision support through a zone-to-zone flow matrix and revenue breakdown. The heatmap reveals trip density patterns, while the horizontal bar chart identifies which pickup zones generate the most revenue—critical intelligence for driver positioning and surge pricing strategies.



Figure 7: Geographic Intelligence: The zone-to-zone heatmap visualizes trip flow intensity, while the revenue breakdown identifies Manhattan as the dominant pickup zone.

4.10.5 Chart Visualizations with Plotly

All charts are rendered using Plotly with a professional dark theme optimized for business presentations:

```

1 # Color theme - Professional dark theme
2 COLORS = {
3     "primary": "#4F46E5",      # Indigo
4     "secondary": "#7C3AED",    # Violet
5     "success": "#10B981",      # Emerald
6     "warning": "#F59E0B",      # Amber
7     "danger": "#EF4444",       # Red
8     "background": "#0F172A",   # Slate 900
9     "card": "#1E293B",        # Slate 800
10    "text": "#F8FAFC",         # Slate 50
11 }
12
13 # Example: Hourly Revenue Bar Chart
14 fig = px.bar(
15     hourly_df,
16     x='hour',
17     y='revenue',
18     title="Hourly Revenue Pattern",
19     color_discrete_sequence=['#8B5CF6']
20 )

```



```

21 fig.update_layout(
22     paper_bgcolor='#1E293B',
23     plot_bgcolor='#1E293B',
24     font_color='#FFFFFF',
25     xaxis_title="Hour of Day",
26     yaxis_title="Revenue ($)"
27 )
28 st.plotly_chart(fig, use_container_width=True)

```

Together, these four interconnected sections form a cohesive business narrative—from high-level executive metrics, through revenue and operational analysis, down to geographic granularity. This storytelling approach ensures that stakeholders at every level can extract the insights most relevant to their decision-making needs.

4.11 Prometheus + Grafana

Real-time observability is critical in distributed big data systems to detect performance bottlenecks, resource exhaustion, and processing failures. Our platform employs a **Prometheus-Grafana** monitoring stack to collect, aggregate, and visualize metrics from the Apache Spark cluster. This combination provides comprehensive visibility into cluster health, job execution, and resource utilization.

4.11.1 Prometheus Configuration

Prometheus is deployed as a containerized time-series database that scrapes metrics from Spark endpoints at regular intervals. The deployment configuration is as follows:

```

1 prometheus:
2   image: prom/prometheus
3   container_name: prometheus
4   volumes:
5     - ./prometheus/prometheus.yml:/etc/prometheus/prometheus.yml
6   ports:
7     - "9090:9090"
8   networks:
9     - taxi-platform

```

Prometheus is configured to scrape metrics from multiple Spark components via the `prometheus.yml` file. We define three distinct scrape jobs, each targeting a different Spark service:

```

1 scrape_configs:
2   - job_name: "spark-master"
3     metrics_path: /metrics/master/prometheus
4     static_configs:
5       - targets: ["spark-master:8080"]
6
7   - job_name: "spark-worker"
8     metrics_path: /metrics/worker/prometheus
9     static_configs:
10      - targets: ["spark-worker:8081"]
11
12   - job_name: "spark-thrift"
13     metrics_path: /metrics/prometheus
14     static_configs:
15      - targets: ["spark-thrift:10000"]

```

Each job is configured with:

- **Scrape Target:** The hostname and port of the Spark service exposing metrics.
- **Metrics Path:** The HTTP endpoint where Prometheus-formatted metrics are served.
- **Static Discovery:** Static target configuration suitable for containerized deployments with known hostnames.

By default, Prometheus scrapes metrics every 15 seconds, providing near-real-time granularity for monitoring dashboards and alerting rules.

4.11.2 Spark Metrics Exporter

To expose internal Spark metrics in Prometheus format, we configure the **PrometheusServlet** sink in Spark's `metrics.properties` file. This lightweight HTTP servlet publishes JVM, executor, and Spark-specific metrics without requiring external exporters:

```
1 # Enable Prometheus sink for all instance types
2 *.sink.prometheusServlet.class=org.apache.spark.metrics.sink.PrometheusServlet
3 *.sink.prometheusServlet.path=/metrics/prometheus
4
5 # Specific paths for Master, Worker, and Driver UIs
6 master.sink.prometheusServlet.path=/metrics/master/prometheus
7 applications.sink.prometheusServlet.path=/metrics/applications/prometheus
8 worker.sink.prometheusServlet.path=/metrics/worker/prometheus
```

This configuration ensures that each Spark component (Master, Worker, and Thrift Server) exposes metrics at distinct endpoints, allowing Prometheus to collect role-specific metrics without collisions. The metrics include:

- **Cluster Metrics:** Number of alive workers, active applications, waiting applications.
- **Resource Metrics:** CPU cores used/free, memory used/free, executor counts.
- **Catalog Metrics:** Hive external catalog operations (cache hits, files discovered, partitions fetched).
- **Code Generation Metrics:** Compilation time percentiles (50th, 75th, 95th, 99th), generated class/method sizes.

4.11.3 Grafana Visualization

Grafana serves as the visualization layer, providing interactive dashboards for real-time monitoring and historical analysis. It is deployed with automatic provisioning to eliminate manual configuration:

```
1 grafana:
2   image: grafana/grafana
3   container_name: grafana
4   volumes:
5     - ./grafana/provisioning/datasources:/etc/grafana/provisioning/datasources
6     - ./grafana/provisioning/dashboards:/etc/grafana/provisioning/dashboards
7   environment:
8     - GF_SECURITY_ADMIN_PASSWORD=admin
9   ports:
10    - "3000:3000"
11   depends_on:
12     - prometheus
13   networks:
14     - taxi-platform
```

Grafana is configured to automatically provision:

1. **Datasource:** Prometheus as the default data source.

2. Dashboards: Pre-built Spark cluster monitoring dashboards.

The Prometheus datasource is defined in `datasource.yml`:

```
1 apiVersion: 1
2
3 datasources:
4   - name: Prometheus
5     type: prometheus
6     access: proxy
7     url: http://prometheus:9090
8     isDefault: true
```

Dashboard provisioning is configured via `dashboards.yml`:

```
1 apiVersion: 1
2
3 providers:
4   - name: 'Spark metrics'
5     orgId: 1
6     folder: 'Imported Dashboards'
7     type: file
8     disableDeletion: true
9     editable: true
10    updateIntervalSeconds: 10
11    options:
12      path: /etc/grafana/provisioning/dashboards
```

This configuration automatically imports dashboard definitions from the mounted directory, ensuring that the monitoring UI is ready immediately after deployment without requiring manual JSON imports.

4.11.4 Spark Cluster Dashboard

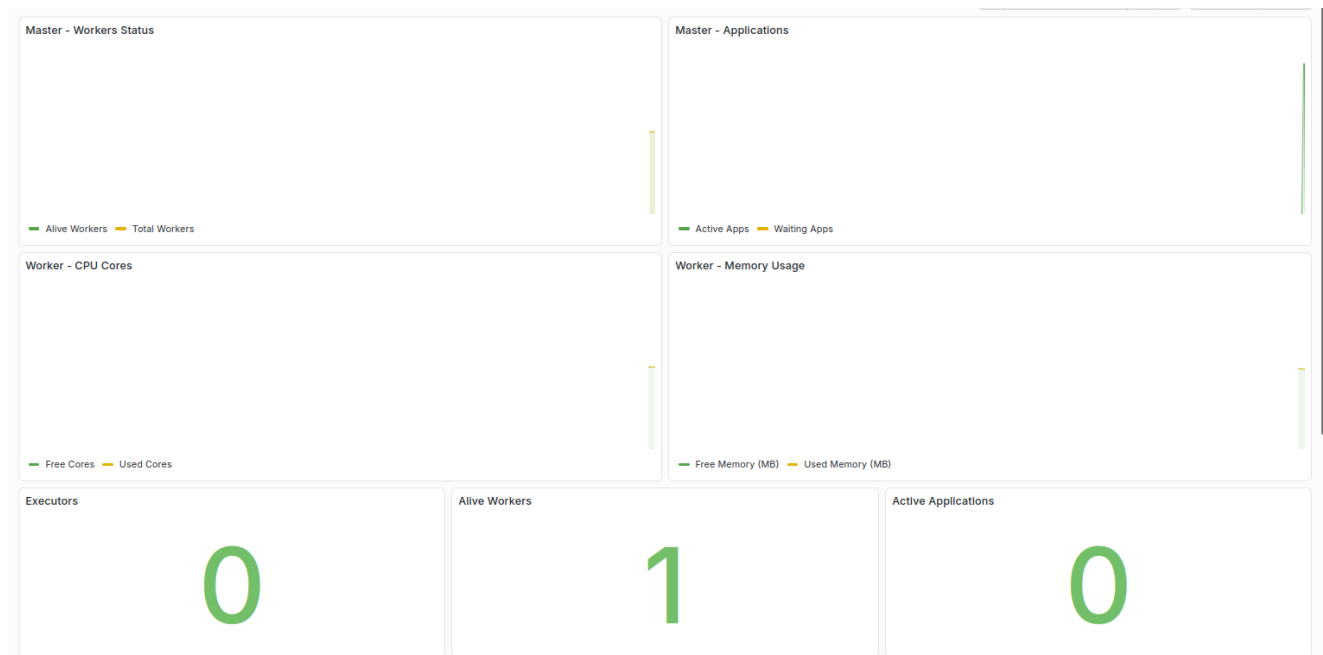


Figure 8: Cluster Health & Resource Utilization Panels

We developed a custom Grafana dashboard (`spark_dashboards.json`) tailored to the operational needs of our platform. The dashboard consists of 11 panels organized into four categories:

Cluster Health Panels

- **Master - Workers Status:** Displays alive workers vs. total registered workers over time, enabling quick detection of node failures.
- **Master - Applications:** Shows active applications and queued (waiting) applications, indicating cluster saturation.

Resource Utilization Panels

- **Worker - CPU Cores:** Stacked area chart showing free vs. used CPU cores, visualizing compute resource availability.
- **Worker - Memory Usage:** Stacked area chart displaying free vs. used memory in megabytes, tracking memory pressure.

Summary Stat Panels

- **Executors:** Current number of active executors across all workers.
- **Alive Workers:** Real-time count of healthy worker nodes.
- **Active Applications:** Number of currently running Spark applications.

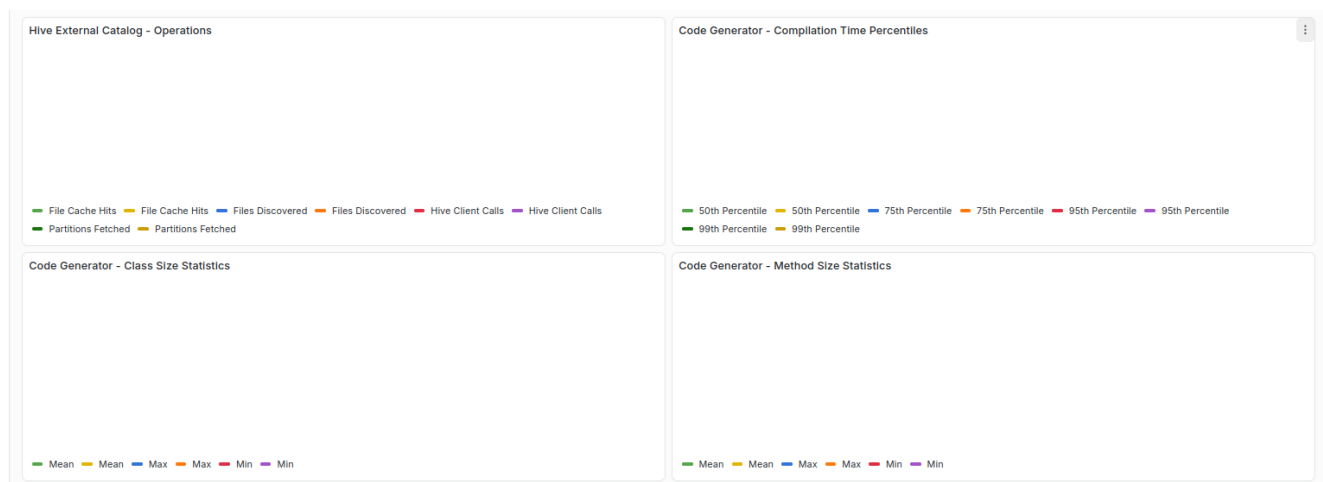


Figure 9: Advanced Metrics Panels

Advanced Metrics Panels

- **Hive External Catalog - Operations:** Tracks metadata operations including file cache hits, files discovered, Hive client calls, and partitions fetched. These metrics are critical for diagnosing slow query planning in Iceberg table scans.
- **Code Generator - Compilation Time Percentiles:** Displays P50, P75, P95, and P99 compilation times for Spark's Whole-Stage Code Generation (WSCG). High percentiles indicate inefficient query plans.
- **Code Generator - Class Size Statistics:** Shows mean, max, and min sizes of generated classes. Extremely large classes may indicate code generation pathologies that degrade performance.
- **Code Generator - Method Size Statistics:** Similar to class size metrics, tracking generated method sizes to detect JVM bytecode limitations (64KB method size limit).

Example PromQL queries used in the dashboard include:

```
1 # Worker CPU Cores
2 metrics_worker_coresFree_Value
3 metrics_worker_coresUsed_Value
4
5 # Master Applications
6 metrics_master_apps_Value
7 metrics_master_waitingApps_Value
8
9 # Hive Catalog Operations
10 metrics_HiveExternalCatalog_fileCacheHits_Count
11 metrics_HiveExternalCatalog_partitionsFetched_Count
12
13 # Code Generation Performance
14 metrics_CodeGenerator_compilationTime_95thPercentile
15 metrics_CodeGenerator_generatedClassSize_Mean
```

The dashboard refreshes every 30 seconds by default, with configurable refresh intervals ranging from 5 seconds to 1 day. Historical analysis is supported through Grafana's time picker, allowing operators to correlate metrics with specific events or deployments.

4.11.5 Monitoring Use Cases

The Prometheus-Grafana stack serves multiple operational purposes:

1. **Performance Debugging:** When Spark jobs exhibit unexpected latency, operators can inspect code generation metrics to identify whether compilation overhead or large generated classes are contributing to slowdowns.
2. **Capacity Planning:** Historical trends in CPU and memory utilization inform scaling decisions, such as adding worker nodes or increasing executor memory allocation.
3. **Failure Detection:** Real-time monitoring of alive workers and active applications enables rapid detection of node crashes or application failures, reducing mean time to recovery (MTTR).
4. **Metadata Optimization:** Hive external catalog metrics reveal inefficiencies in metadata fetching patterns, guiding optimizations such as partition pruning or catalog caching strategies.

This monitoring infrastructure provides the observability foundation necessary to operate a production-grade big data platform, enabling data engineers to maintain high availability and performance at scale.

5. Conclusion

Through this project, our team has successfully explored, researched, and built a comprehensive Big Data pipeline using the Kappa Architecture. By integrating core technologies such as Kafka for ingestion, HDFS for storage, Spark for processing, and Cassandra for the serving layer, we created a system capable of handling meteorological data in both real-time and batch modes.

We successfully implemented the "Medallion" architecture (Bronze, Silver, Gold), ensuring that raw data is systematically cleaned, aggregated, and transformed into actionable Machine Learning insights. This project not only reinforced our technical skills in distributed systems but also demonstrated the practical application of data engineering principles to solve real-world problems.

While the current system is functional, we aim to further develop the pipeline to meet more diverse user needs in the future.

6. Acknowledgments

To complete this report, we would like to express our deepest gratitude to our instructor, Dr. Tran Viet Trung. His invaluable and inspiring lectures in the course *"Big Data Analysis and Processing"* provided us with the solid theoretical foundation and practical guidance necessary to navigate these complex technologies. We sincerely appreciate the knowledge and enthusiasm he has shared with us throughout this semester.

6.1 Lessons Learned

During development and testing of the NYC Taxi streaming pipeline we encountered a number of practical issues that informed our design choices.

6.1.1 Lesson 1: Container Networking & Listener Configuration

Problem: Services inside Docker require service-name addressing; using 'localhost' inside a container breaks cross-service communication and can cause Kafka bootstrap/advertised listener mis-configurations.

Solution: Use a shared Docker network (see 'docker-compose.yml') and advertise listeners appropriate to the client context: from inside containers the bootstrap address is `kafka:29092`, while from the host it is `localhost:9092`. Verify 'KAFKA_ADVERTISED_LISTENERS' and test connectivity from both host and container contexts.

Key Takeaway: Prefer service names for container-to-container communication and confirm advertised listeners for host-facing tooling (UI, local Spark jobs).

6.1.2 Lesson 2: Checkpointing, Watermarks, and Exactly-once Safety

Problem: Streaming jobs (Spark → Iceberg) must avoid duplicates and ensure progress is tracked across restarts; failing to persist offsets/checkpoints leads to reprocessing or data loss.

Solution: Use Spark structured streaming checkpoints (`checkpointLocation` to S3/MinIO) and maintain a small Iceberg 'watermarks' table (e.g., 'last_watermark_ts') to coordinate incremental Silver/Gold runs. Emit per-batch metrics (`inputRows`, processing latency) to help detect regressions.

Key Takeaway: Durable checkpoints + watermark bookkeeping provide practical, testable guarantees for incremental pipelines.

6.1.3 Lesson 3: Schema Evolution with Iceberg

Problem: Evolving payload formats (new nullable columns, renamed fields) can break consumers if uncoordinated.

Solution: Use Iceberg schema evolution features: add new columns as nullable, prefer additive changes, and provide migration runs to backfill historic data when necessary. Validate consumer reads post-migration using the Thrift server or 'spark.sql' queries.

Key Takeaway: Plan schema changes, communicate with downstream consumers, and test evolution on a small staging dataset before applying to production.

7. Member Contribution

Member Name	Assigned Tasks & Responsibilities	Contribution
Tran Huu Dao	• Deployed and configured Debezium (PostgreSQL CDC → Kafka). • Set up the Spark Thrift Server (HiveServer2) for JDBC/ODBC access. • Developed Streamlit dashboards with PyHive connectivity to the Lakehouse.	20%
Dao Vu Tien Dat	• Implemented Batch streaming from Kafka to Iceberg. • Developed real-time streaming in Apache Flink.	20%
Ngo Quang Duc	• Implemented Spark Processing Script for Medallion Architecture. • Implement the incremental processing strategy. • Design the OLAP Data Model.	20%
Vu Nhu Duc	• Developed and orchestrated a daily Lakehouse ETL pipeline using Airflow DAGs • Configured programmatic Spark connectivity by Airflow API	20%
Ta Tuan Hai	• Data streaming into OLTP Layer (PostgreSQL). • Implementing a monitoring dashboard for Sparks using Grafana and Prometheus. • Design presentation slide.	20%

Table 2: Breakdown of team member responsibilities and contribution percentage.